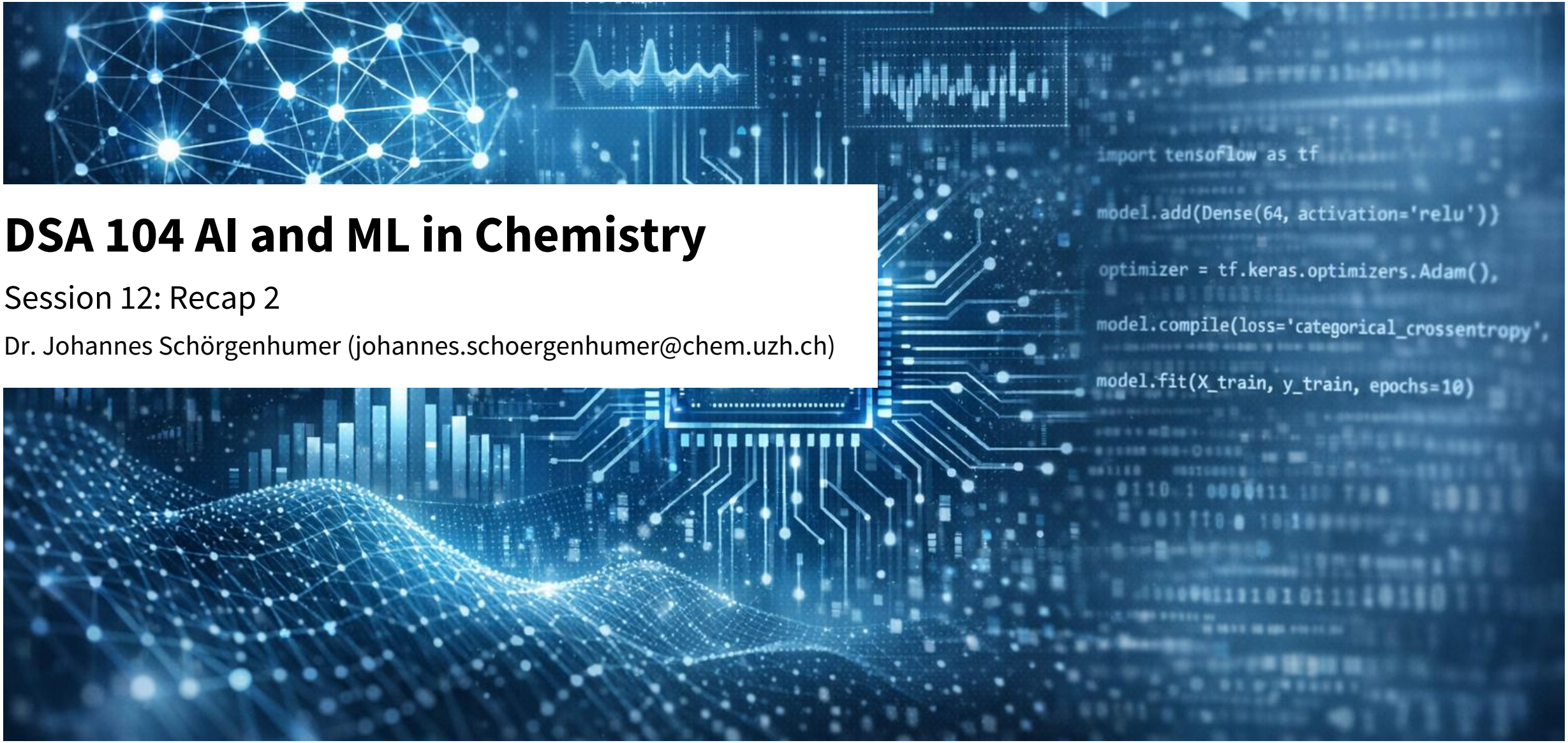
Universität Zürich[UZH]

Department of Chemistry



DSA 104 AI and ML in Chemistry

Session 12: Recap 2

Dr. Johannes Schörgenhumer (johannes.schoergenhumer@chem.uzh.ch)

Points for today

Discuss Assignment 4

Q&A, Feedback

Choosing parameters for NN

Quiz time

Light content: Time series

Feedback on exercises

Approximate time demand on exercises 3 and 4?

Technical issues (environment, compilation time, etc.)

NN Building guide – recap:

Don’t over-engineer!

Use **pretrained models** or **standard backbones**:

— CNNs, RNNs, Transformers, etc

If you do build from scratch (**MLP** / **FNN**):

— **Depth** for standard tasks: 2-4 hidden layers

— More complex tasks: 6-8 hidden layers

— > 10 layers: vanishing gradients, training issues (unless transformers, or residual connections used)

— If many layers: use residuals

— **Width**: start with 2x to 8x number of features (64-512 is a good starting point)

— Double or half on optimization

NN Building guide – recap:

Unless specific tasks, ReLU family good starting point for **activation function**:

— ReLu

— LeakyReLU

— GELU

Use appropriate **loss functions**, e.g.

— BCEWithLogitsLoss (binary classification)

— MSELoss (Regression)

Optimisers:

— AdamW generally a good default (e.g. with a **learning rate** of 0.001)

— If loss oscillates: lr too high

— If loss barely changes: lr to low

NN Building guide – recap:

Check for overfitting / underfitting!

In case of **overfitting**:

- Reduce size / complexity
- Introduce **regularisation**:
  - **Dropout** for fully-connected nets: start with 0.1 to 0.3
  - **Weight decay**: use default (1e-4) with **AdamW**
  - **Batch normalisation**: use between layers for stability (can allow higher lr)

Batch size, e.g. 64 as start (depends on memory and task)

- Large: more stable gradient
- Small: better generalisation

NN Building guide – starting recipe in numbers:

— Architecture: Input – Hidden(128) – Hidden(256) – Hidden(128) – Output

— Activation: ReLU or GELU

— Optimiser: AdamW(lr=1e-3)

— Optimiser: Parameters

— Regularisation: weight_decay=1e-4

— Dropout: 0.1

— Normalisation: e.g. BatchNorm

— Batch size: 64

— **Early stopping!**

NN Building

Look at different options (e.g. regularisation) in the *pytorch_NNbuilding* notebook.

Time Series

A sequence of observations ordered in time, e.g. sensor data, website traffic, …: $\boldsymbol{x_1}, \boldsymbol{x_2}, \boldsymbol{x_3}, \boldsymbol{x_4}, \ldots \boldsymbol{x_t}$

Key characteristic:

— **Temporal dependency**: past observations influence future values

Typical goals:

— Forecasting

— Anomaly detection

— Classification of sequences

Time Series – key characteristics

Key properties:

— **Trend**: Long term increase/decrease

— **Seasonality**: Repeating patterns in regular intervals

— **Cycles**: Longer irrecular fluctuations

— **Noise**: Random variation not explained by the signal

Model: $y_t = Trend + Seasonality + Noise$

Stationarity for a time series is given if…

— Mean is constant

— Variance is constant

Typical transformations:

— Differencing

— Detrending ($y'_t = y_t - y_{t-1}$)

— Log transformations (e.g. FT)

— Seasonal decomposition

Time Series – how to treat the data:

Python datetime objects may be very useful!

Feature engineering:

— Lag features: past observations used as predictors ($x_{t-1}, x_{t-2}, x_{t-3}$)

— Rolling statistics: e.g. rolling mean ($rolling\ mean_t = \frac{1}{k}\sum_{i=0}^{k-1} x_{t-i}$)

— Calendar features: e.g. hour, day of the month

Data splits:

— Random splits don't work

— Use temporal slice instead!

Models:

— Classical models, e.g. AutoRegressive, Moving Average

— ML: Treat as regression!

— DL: NN, Transformers

First look at Time Series

Have a look at the provided notebook (*timeseries.ipynb*) and…

…discuss what the individual steps do

…find some acceptable parameters

…how can you actually implement forecast of the next value? (Recursive vs. direct forecasting!)